

1 O návrhových vzorech

Když máte jen kladivo, všechno vypadá jako hřebík. Berličky pro slabý programovací jazyk - Například vzor Strategy lze v mnoha moderních programovacích jazycích realizovat prostě pomocí anonymní (lambda) funkce

1.1 Rozdělení podle záměru podle záměru

1.1.1 Creational patterns

Starají se o vytváření objektů.

- Singleton
- Factory Method
-
-
-
-
-

1.1.2 Structural patterns

Vysvětlují skládání/vztah objektů a tříd do větších celků.

- Flyweight
-
-
-
-

1.1.3 Behavioral patterns

Popisují komunikaci a rozdělení zodpovědnosti mezi objekty.

- Strategy
-
-
-
-
-

2 Paterny

2.1 Factory Method

Umožňuje vytvářet objekty pomocí metody namísto konstruktoru. Vytvářené objekty sdílí stejný interface, což umožňuje metodě vracet různé objekty. Metoda buďto může vracet různé objekty podle dodaných parametrů, anebo můžeme od Factory (třída, která obsahuje faktory metodu) dědit další factory, které vrací různé objekty. Používat interface pro továrnu umožňuje ji lehce vyměnit při změně kódu anebo při testování. Umožňuje mít veškerý kód pro vytváření objektu na jednom místě a lehce měnit jaký objekt bude vytvořen na více místech v kódu.

Výhody:

- Snižuje propojení mezi třídami.
- Single Responsibility Principle.
- Kód je jednoduše rozšiřitelný. Můžeme přidávat nové produkty a továrny bez změny předchozího kódu. Knihovna používající Factory method je mnohem více rozšiřitelná.
- Snižuje duplikaci kódu okolo vytváření objektu.
- Umožňuje vracet už existující instanci namísto vytváření nové.

Nevýhody:

- Továrna může vracet pouze objekty dědící daný interface.
- Vytváření velkého množství interfaců a podtříd.

2.1.1 Relations with Other Patterns

- Factory method se vyvíjí do Abstract Factory, Prototype, Builder.
- Factory method se používá k vracení různých Iterátorů.
- Factory method je druh Template Method. Zároveň Factory může být součástí větší Template Method.

2.1.2 More info

- <https://refactoring.guru/design-patterns/factory-method>

2.2 Singleton

Používá se pokud je v programu potřeba aby existovala jediná instance určité třídy. K této třídě také poskytuje globální přístup. Tím porušuje zákon Single Responsibility, jelikož řeší 2 věci najednou. Často se používá u konfigurací, přístupu k souboru nebo databázi. Jedná se o globální stav a všechny objekty na které obsahuje reference jsou také globální stav => je těžké je mockovat v testech. Podobné problémy jako globální proměnné. Singleton (velké S) je patern, zatímco singleton (malé s) je třída která by měla mít pouze jednu instanci.

Výhody:

- Lazy vytváření.
- Pouze jeden existující objekty.
- Jednoduché řešení (na začátku).

Nevýhody:

- Poskytuje globální přístup k proměnným, praktické, ale velmi nebezpečné. Může zakrývat špatný návrh kódu. Může vytvářet zbytečně velké závislosti mezi třídami.
- Vyžaduje úpravu pro multi-thread safety.
- Pokud možno je lepší používat dependency injection.
- Špatně se testuje. Jelikož je singleton měnící se statická hodnota, tak si zachová změny mezi testy (Singletons are pathological liars.). Je potřeba ho mezi testy resetovat mezi testy, testy také nemohou běžet paralelně. Stejně tak mockování může být obtížné.
- Zvětšuje propojení mezi třídami.
- Může skrývat co se děje v kódu a skrývat propojení mezi třídami.

Častou chybou je následující kód, ve kterém se snažíme vytvořit třídu CreditCard. Jenže třída CreditCard nefunguje pokud nejdříve nespustíme OfflineQueue.init() a ta potřebuje nejdříve spustit CreditCardProcessor.init();. V kódu nic ne naznačuje, že by na sobě třídy byly závislé jelikož je to skryté.

```
CreditCardProcessor.init();
OfflineQueue.init();
CreditCard c = new CreditCard( "1234 5678 9012 3456", 5, 2008);
c.charge(100);
Database.init();
```

Je lepší závislosti ukázat tím, že je budeme vyžadovat předat k operaci.

```
Database db = Database();
OfflineQueue q = OfflineQueue(db);
CreditCardProcessor ccp = new CreditCardProcessor(q);
CreditCard c = new CreditCard( "1234 5678 9012 3456", 5, 2008);
c.charge(ccp, 100);
```

celý příklad

2.2.1 Relations with Other Patterns

2.2.2 More info

- <https://www.youtube.com/watch?v=-FRm3VPheI>
- <https://testing.googleblog.com/2008/08/by-miko-hevery-so-you-join-new-project.html>

3 Anti-Paterny

3.1 spaghetti code

3.2 lava flow

3.3 accidental complexity

3.4 God Object

3.5 hard coding

3.6 magic numbers

3.7 Too many responsibilities

4 Zdroje

4.1 Použité zdroje

- Prezentace ANV 2025
- <https://testing.googleblog.com/2008/08/by-miko-hevery-so-you-join-new-project.html>
- <https://www.youtube.com/watch?v=-FRm3VPheI>
- <https://refactoring.guru/design-patterns/factory-method>
- https://en.wikipedia.org/wiki/Software_design_pattern
- more refactoring.guru ... more wikipedia

4.2 Doporučené zdroje

Zdroje doporučené učiteli, použitými zdroji anebo jinak užitečné zdroje na toto téma které zatím nebyli plně vyčerpány.

- Design Patterns: Elements of Reusable Object-Oriented Software
- Patterns and Antipatterns - Andrew Koenig
- Roberta C. Martina Clean Code: A Handbook of Agile Software Craftsmanship